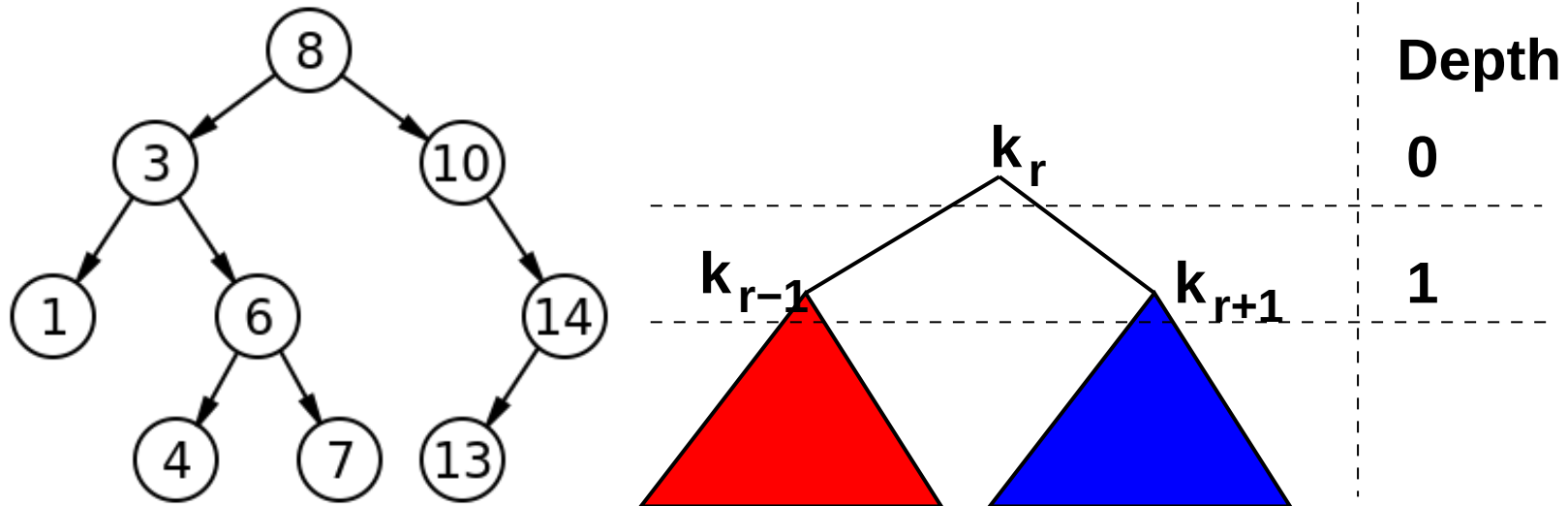


Chapter 5

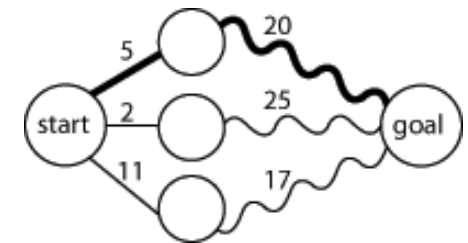
Dynamic Programming



Dynamic Programming

- **Simplex for LP:** Greedy algorithm, makes a **locally optimal** choice.
- For many problems, we need a different approach called **Dynamic Programming**
- Finds solutions for problems with lots of **overlapping sub-problems**. Essentially, we try to solve each sub-problem **only once**.
- **Optimal substructure:** optimal solutions of **subproblems** can be used to find the optimal solutions of the **overall problem**.

Example: Finding the shortest path in a graph.



Dynamic Programming

Typically, a dynamic programming solution is constructed using a **series of steps**:

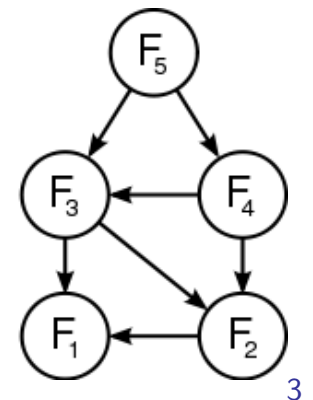
1. Characterise the **structure** of an optimal solution.
2. **Recursively** define the value of an **optimal solution**.
3. Compute the value of an optimal solution in a **bottom-up** ($\rightsquigarrow$ iteration) or **top-down** ($\rightsquigarrow$ recursion) fashion. That is, build it from the results of **smaller solutions** either **iteratively** from the bottom or **recursively** from the top.

A Simple Example: Fibonacci numbers

Fibonacci sequence: The n -th number is the sum of the previous two. This can be implemented using a simple recursive algorithm:

```
function FIBONACCI( $n$ )  
    if  $n = 0$  then  
        return 0  
    if  $n = 1$  then  
        return 1  
    return FIBONACCI( $n - 1$ ) + FIBONACCI( $n - 2$ )
```

Problem: Overlapping sub-problems: Computing
FIBONACCI($n - 1$) overlaps FIBONACCI($n - 2$)
 $\rightsquigarrow$ exponential time complexity!



A Simple Example (2)

Define **map object** m , maps each instance of FIBONACCI that has already been calculated to its result.

Modified recursion requires only $O(n)$ time:

```
var  $m$ ;  $m[0] = 0$ ;    $m[1] = 1$   
function FIBONACCI( $n$ )  
    if  $m$  does not contain key  $n$   
         $m[n] = \text{FIBONACCI}(n - 1) + \text{FIBONACCI}(n - 2)$   
    return  $m[n]$ 
```

Or define array f and use **iteration**: $f[0] = 0$, $f[1] = 1$.

```
FIBONACCI( $n$ )  
    for  $i = 2$  upto  $n$  step 1 do  
         $f[i] = f[i - 1] + f[i - 2]$   
    return  $f[n]$ 
```

Another Example: Optimal Binary Search Trees

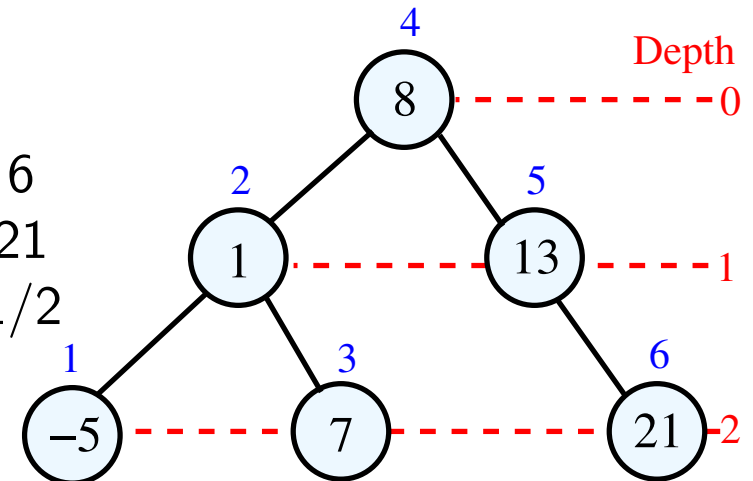
BST: Tree where the key values are stored in the nodes, and the keys are ordered lexicographically.

For each internal node all keys in the left subtree are less than the keys in the node, and all the keys in the right subtree are greater.

Knowing the probabilities of searching each one of the keys makes it easy to compute the expected cost of accessing the tree.

An **OBST** is a BST with *minimal expected search costs*.

Index	1	2	3	4	5	6
Keys	-5	1	7	8	13	21
Probabilities	1/8	1/32	1/32	1/16	1/4	1/2



OBST

- **Keys** $k_1, \dots, k_n$ in lexicographical order,
- **Probabilities** of accessing keys $p_1, \dots, p_n$.
- **Depth** $D_T(k_m)$ of node k_m in tree T . $D_T(\text{root}) = 0$
- T^{ij} : tree constructed from keys $k_i, \dots, k_j$
- **Costs**: number of comparisons done in a search.
- **Expected costs**: expected number of comparisons done during search in tree, given the access probabilities p_i

OBST: Expected costs

Definiton of expected costs of tree constructed from keys $k_i, \dots, k_j$:

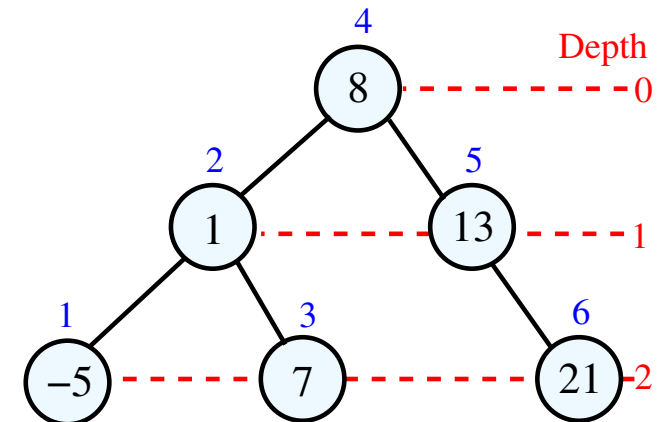
$$C_{i,j} := E[\text{cost}(T^{ij})]$$

$$= \sum_{\text{all keys in } T^{ij}} \text{prob. of key} \times (\text{depth of key} \overset{\substack{\text{one comparison for root} \\ +1}}{+1})$$

$$= \sum_{m=i}^j p_m (D_T(k_m) + 1)$$

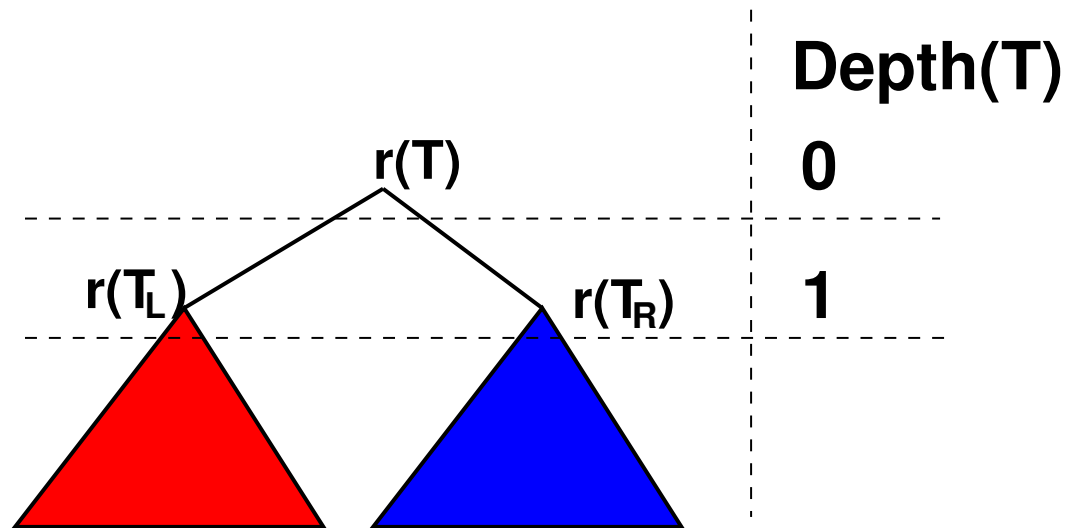
Index	1	2	3	4	5	6
Keys	-5	1	7	8	13	21
Probabilities	1/8	1/32	1/32	1/16	1/4	1/2

$C_{1,6} = 1 \cdot 1/16 + 2 \cdot (1/32 + 1/4) + 3 \cdot (1/8 + 1/32 + 1/2)$
 $= 85/32$



OBST

- **Key observation:** each subtree of an optimal tree is itself optimal (replacing a subtree with a better one lowers the costs of entire tree)
- Consider tree T^{ij} with root node $r(T) = k_r$.



Expected costs of tree $T = T^{ij}$

$$\begin{aligned}
 C_{i,j} &= \sum_{m=i}^j p_m (D_T(k_m) + 1) \\
 &= \sum_{m=i}^{r-1} p_m (D_T(k_m) + 1) + p_r + \sum_{m=r+1}^j p_m (D_T(k_m) + 1) \\
 &= \underbrace{\sum_{m=i}^{r-1} p_m ((D_{T_L^r}(k_m) + 1) + 1)}_{C(\text{left subtree})|_{\text{root}=r}} + \underbrace{p_r}_{\text{root}} + \underbrace{\sum_{m=r+1}^j p_m ((D_{T_R^r}(k_m) + 1) + 1)}_{C(\text{right subtree})|_{\text{root}=r}} \\
 &= C(T_L^r) + \sum_{m=i}^{r-1} p_m + p_r + C(T_R^r) + \sum_{m=r+1}^j p_m \\
 &= C_{i,r-1} + C_{r+1,j} + \sum_{m=i}^j p_m, \quad i \leq r \leq j.
 \end{aligned}$$

OBST: algorithm

Recursive algorithm:

- consider every node as being the root
- split rest of the keys into left and right subtrees and recursively calculate their costs.

$$C_{i,i} = p_i$$

$$C_{i,j} = 0 \quad \forall j < i \quad (\text{tree with no nodes})$$

$$C_{i,j} = \sum_{m=i}^j p_m + \min_{i \leq r \leq j} [C_{i,r-1} + C_{r+1,j}]$$

Use **memoization** to avoid solving the same problem over and over.
Or use **iterative** algorithm.

DP for an OBST

- Precompute $P_{ij} = \sum_{m=i}^j p_m$.
- Fill C -matrix by diagonals (start with main diagonal, move up-right)
- Store “winning” root index in matrix R

$$(C)_{ij} = \begin{array}{c|cccccc} & 1 & 2 & 3 & 4 & 5 & 6 \\ \hline 1 & \frac{1}{8} & \frac{3}{16} & \frac{9}{32} & \frac{15}{32} & \frac{31}{32} & \frac{63}{32} \\ 2 & 0 & \frac{1}{32} & \frac{3}{32} & \frac{7}{32} & \frac{19}{32} & \frac{47}{32} \\ 3 & & 0 & \frac{1}{32} & \frac{1}{8} & \frac{15}{32} & \frac{21}{16} \\ 4 & & & 0 & \frac{1}{16} & \frac{3}{8} & \frac{19}{16} \\ 5 & & & & 0 & \frac{1}{4} & 1 \\ 6 & & & & & 0 & \frac{1}{2} \end{array} = \frac{1}{32} \begin{pmatrix} 4 & 6 & 9 & 15 & 31 & 63 \\ & 1 & 3 & 7 & 19 & 47 \\ & & 1 & 4 & 15 & 42 \\ & & & 2 & 12 & 38 \\ & & & & 8 & 32 \\ & & & & & 16 \end{pmatrix}$$

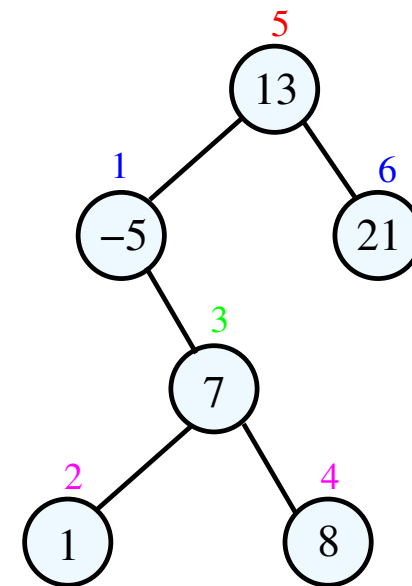
- Find tree by backtracking: start in upper right corner $R_{1,n}$
 $\rightsquigarrow$ **root of full tree**, say root = k .
Right subtree: proceed with $R(k+1, n)$
 $\rightsquigarrow$ **root of right subtree** $T_{k+1,n}$, say $R(k+1, n) = r$.
Draw edge $k \rightarrow r$.
Left subtree: $R(1, k-1) = l \rightsquigarrow$ **root of left subtree**, edge $k \rightarrow l$.
Recurse.

Computations

In our case:

Index	1	2	3	4	5	6
Keys	-5	1	7	8	13	21
Probabilities	1/8	1/32	1/32	1/16	1/4	1/2

$$R = \begin{pmatrix} 1 & 1 & 1 & 1 & 5 & 5 \\ & 2 & 2 & 3 & 5 & 6 \\ & & 3 & 4 & 5 & 6 \\ & & & 4 & 5 & 6 \\ & & & & 5 & 6 \\ & & & & & 6 \end{pmatrix}$$



$$\begin{aligned} E[\text{cost}] &= 1 \cdot 1/4 + 2 \cdot (1/2 + 1/8) + 3 \cdot (1/32) + 4 \cdot (1/16 + 1/32) \\ &= 1/32[8 + 2(16 + 4) + 3 + 4(2 + 1)] = 63/32. \end{aligned}$$

Chained Matrix Multiplication

- Problem: Given a series of n matrices (of appropriate sizes) to multiply:
Compute $A_1 A_2 \cdots A_n$.
- Determine **where to place parentheses** in order to **minimize the number of multiplications**.
- **Matrix multiplication is associative:** $A_1(A_2 A_3) = (A_1 A_2) A_3$,
so all placements give same result.
- Formal problem:
Given a sequence of matrices $A_1, A_2, \dots, A_n$,
insert parentheses so that the **product of the matrices** needs the **minimal number of multiplications**.

Number of Multiplications / Parenthesizations

- **Multiplying $i \times j$ and a $j \times k$ matrix: ijk multiplications:**
each element requires j multiplications, and there are ik elements.
- Given the matrices A_1, A_2, A_3, A_4 ,
assume the dimensions of $A_1 = d_0 \times d_1$, $A_2 = d_1 \times d_2$ etc.
- Below are the five possible parenthesizations of these arrays, along with the number of multiplications:

$$(A_1 A_2)(A_3 A_4) : d_0 d_1 d_2 + d_2 d_3 d_4 + d_0 d_2 d_4$$

$$((A_1 A_2) A_3) A_4 : d_0 d_1 d_2 + d_0 d_2 d_3 + d_0 d_3 d_4$$

$$(A_1 (A_2 A_3)) A_4 : d_1 d_2 d_3 + d_0 d_1 d_3 + d_0 d_3 d_4$$

$$A_1 ((A_2 A_3) A_4) : d_1 d_2 d_3 + d_1 d_3 d_4 + d_0 d_1 d_4$$

$$A_1 (A_2 (A_3 A_4)) : d_2 d_3 d_4 + d_1 d_2 d_4 + d_0 d_1 d_4$$

Number of Multiplications / Parenthesizations

- The number of parenthesizations is at least $T(n) \geq T(n-1) + T(n-1)$, since the number with the first element removed is $T(n-1)$, which is also the number with the last removed
- Thus the **number of parenthesizations is $\Omega(2^n)$**
- The number is actually
$$T(n) = \sum_{k=1}^{n-1} T(k)T(n-k)$$
- This is because the original product can be split into 2 subproducts in k places. Each split is to be parenthesized optimally.

Characterizing the Optimal Parenthesization

- **optimal parenthesization of $A_1 A_2 \cdots A_n$** breaks the product into two expressions, each of which is parenthesized or is a single array.
- **Assume the break occurs at position k .**
- **Principle of Optimality:** In the optimal solution for all n matrices, the solution of the sub-problem $A_1 A_2 \cdots A_k$ must be optimal.
Why? Otherwise, we could improve the optimal “full” solution by improving the solution of the sub-problem.
- **Use Dynamic Programming:** Consider a recursive solution, then improve it's performance with memoization or by rewriting bottom up.
- Dimensions of matrix A_i is $d_{i-1} \times d_i$
 $\rightsquigarrow$ the product $A_i A_{i+1} \cdots A_j$ is a matrix of size $d_{i-1} \times d_j$.

Recursive Solution

- $M_{ij} = \#(\text{multiplications})$ in best way to parenthesize arrays $A_i \cdots A_j$.
- $M[i, i] = 0$, since no product is required.
- Optimal solution of $A_i A_{i+1} \cdots A_j$ must break at some k , $i \leq k < j$
- $M[i, j] = M[i, k] + M[k + 1, j] + d_{i-1}d_kd_j$
- $$M[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{M[i, k] + M[k + 1, j] + d_{i-1}d_kd_j\} & \text{if } i < j \end{cases}$$
- **This is easily expressed as a recursive function,**
but with exponential complexity.

Efficient Computation

- We must find a way to **calculate this bottom up.**
Which values does $M[i, j]$ depend on?
- Consider a $n \times n$ matrix of values $M[i, j]$:
Diagonal is 0, build upper triangular table by diagonals.
- Example: Array dimensions:

A_1: 2 x 3

A_2: 3 x 5

A_3: 5 x 2

A_4: 2 x 4

A_5: 4 x 3

Array sizes: $A_i = d_{i-1}$ by d_i :

j	0	1	2	3	4	5
d_j	2	3	5	2	4	3

Example Showing Tables and Calculations

M_{ij} = #(multiplication) in best way to parenthesize arrays $A_i \cdots A_j$

i, j	1	2	3	4	5
1	0	30	42	58	78
2		0	30	54	72
3			0	40	54
4				0	24
5					0

Calculating M_{25} = number of multiplies in the optimal way to parenthesize $A_2A_3A_4A_5$:

$$M_{25} = \min \begin{cases} M_{22} + M_{35} + d_1 d_2 d_5, \\ M_{23} + M_{45} + d_1 d_3 d_5, \\ M_{24} + M_{55} + d_1 d_4 d_5. \end{cases}$$

Example Showing Tables and Calculations

$$= \min \begin{cases} 0 + 54 + 45 = 99 \\ 30 + 24 + 18 = 72 \\ 54 + 0 + 36 = 90 \end{cases}$$

$$= \min \begin{cases} (A_2)(A_3A_4A_5) \\ (A_2A_3)(A_4A_5) \\ (A_2A_3A_4)(A_5) \end{cases}$$

Optimal locations for parentheses:

i, j	1	2	3	4	5
1		1	1	3	3
2			2	3	3
3				3	3
4					4
5					

DP for Aligning Biological Sequences

Histone H1 (residues 120-180)

HUMAN	KKASKPKKAASKAPT	KKPKATPVKKAKKK	LAATPKKAKKPKT	TVKAKPVKASKPKKAKPVK
CHIMP	KKASKPKKAASKAPT	KKPKATPVKKAKKK	LAATPKKAKKPKT	TVKAKPVKASKPKKAKPVK
MOUSE	KKAAPKKAASKAP	SKKPKATPVKKAKKK	PAATPKKAKKPKV	VKVPVKASKPKKAKTVK
RAT	KKAAPKKAASKAP	SKKPKATPVKKAKKK	PAATPKKAKKPKI	VKVVPVKASKPKKAKPVK
COW	KKAAPKKAASKAP	SKKPKATPVKKAKKK	PAATPKKTKKPKT	TVKAKPVKASKPKKTKPVK
	*** :	***** :	***** :	***** :
NON-CONSERVED AMINO ACIDS	Conservative	Conservative	Non-conservative	Conservative
			Non-conservative	Semi-conservative
			Conservative	Non-conservative

By

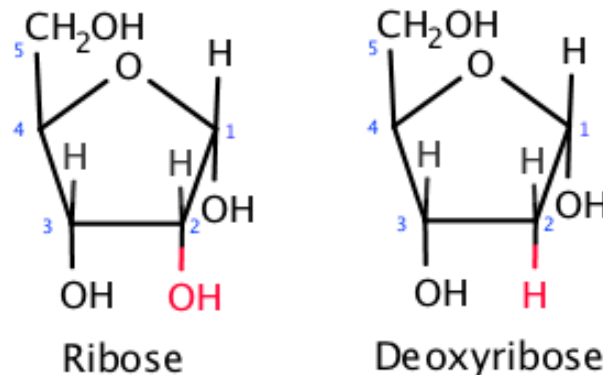
Thomas Shafee - Own work, CC BY 4.0, <https://commons.wikimedia.org/w/index.php?curid=37188728>

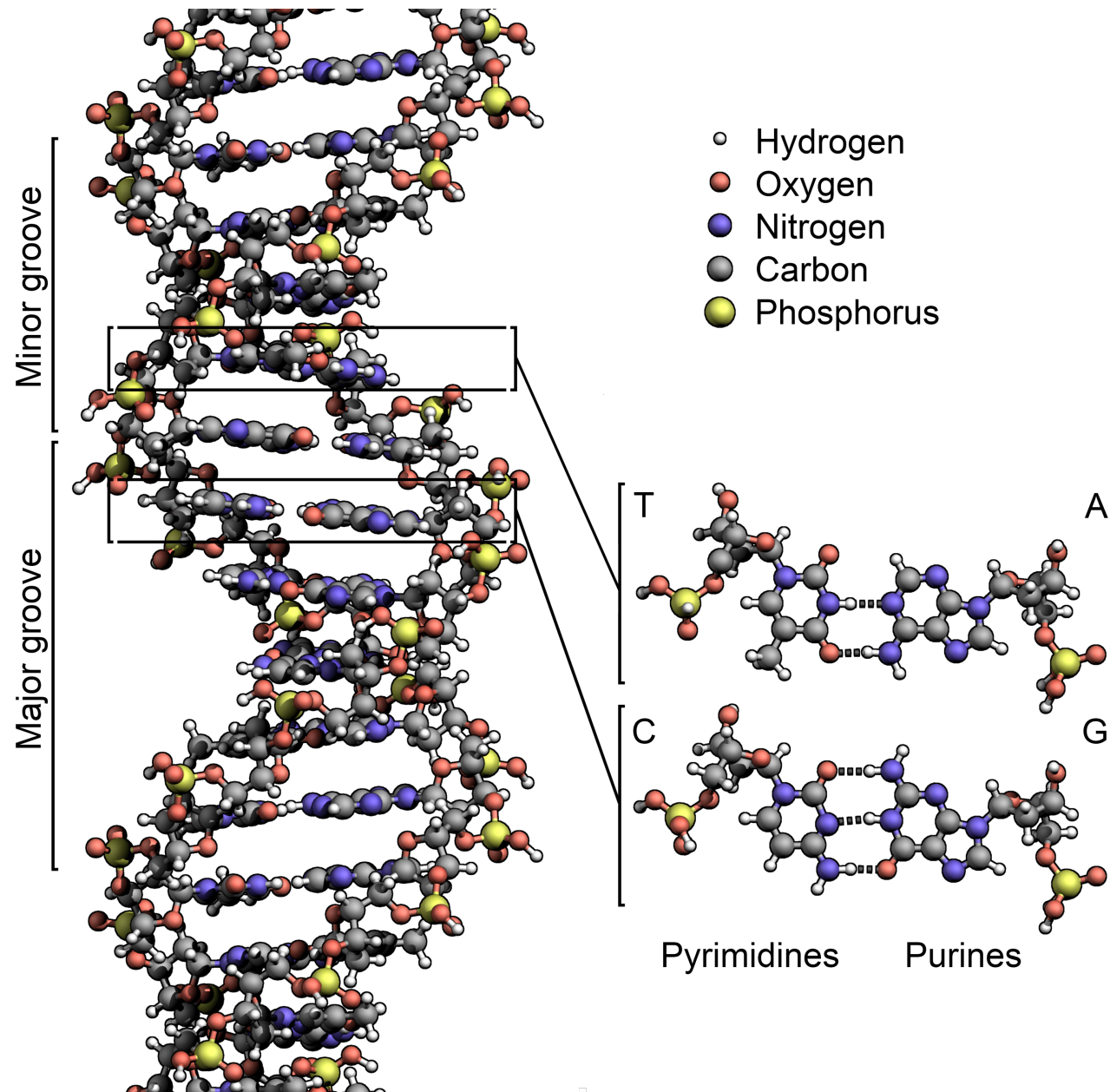
Genetics and the DNA

- Genetics as a natural science started in 1866: **Gregor Mendel** performed experiments that pointed to the existence of **biological elements called genes.**
- **Deoxy-ribonucleic acid (DNA)** isolated by **Friedrich Miescher** in 1869.
- 1944: Oswald Avery (and coworkers) identified DNA as the major carrier of genetic material, **responsible for inheritance.**

Ribose: (simple) sugar molecule, deoxy-ribose $\rightsquigarrow$ loss of oxygen atom.

Nucleic acid: overall name for DNA and RNA (large biomolecules). Named for their initial discovery in nucleus of cells, and for presence of phosphate groups (related to phosphoric acid).





By Zephyris - Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=15027555>

Replication of DNA

Biological process of producing two replicas of DNA from one original DNA molecule.

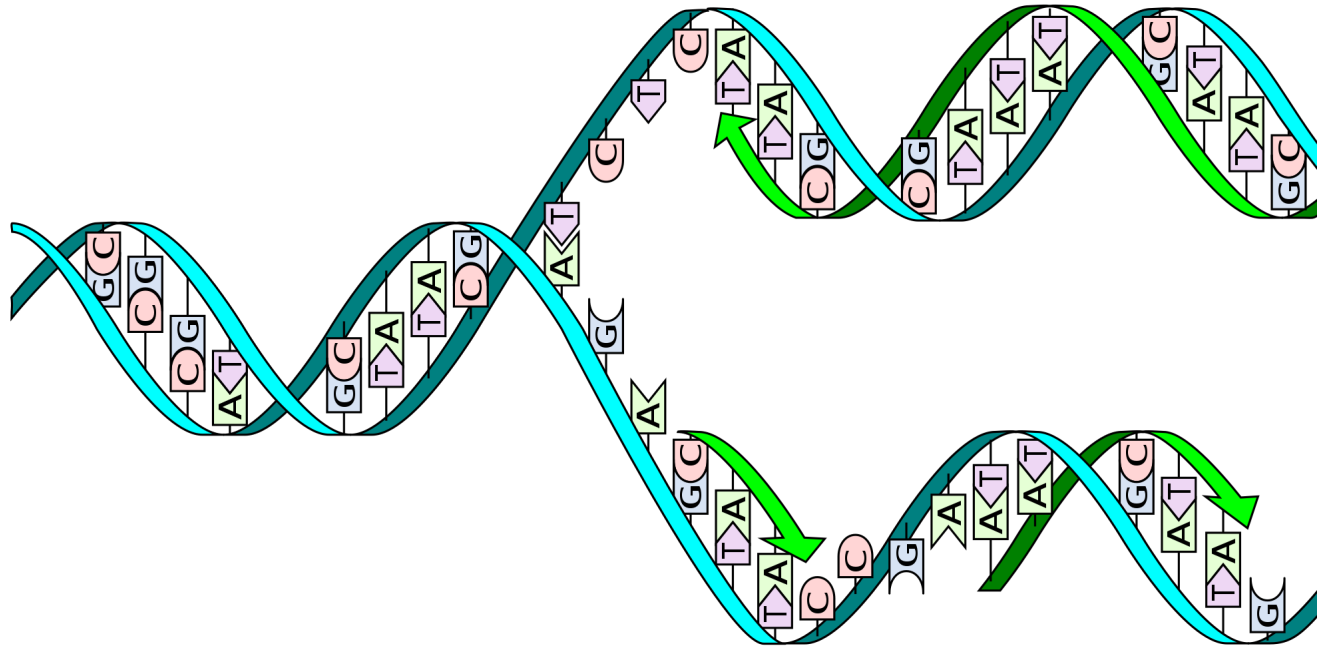
Cells have the distinctive property of division

⇒ DNA replication is most essential part for **biological inheritance**.

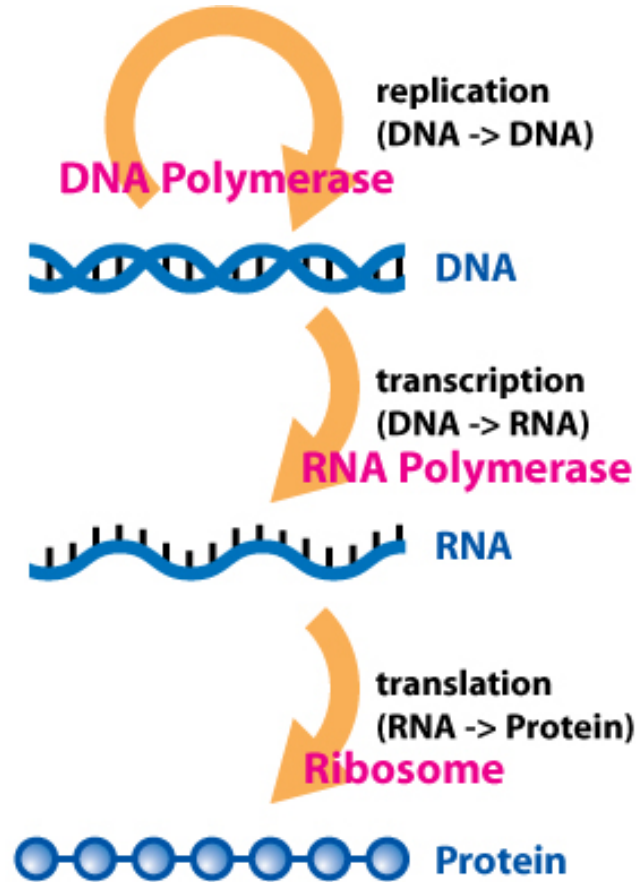
Unwinding ⇒ single bases exposed on each strand.

Pairing requirements are **strict** ⇒ single strands are templates for re-forming **identical** double helix (up to **mutations**).

DNA polymerase: enzyme that catalyzes the synthesis of new DNA.



The Central Dogma



Transcription: making of an RNA molecule from DNA template.

Translation: construction of amino acid sequence from RNA.

⇒ Almost no exceptions (↪ retroviruses)

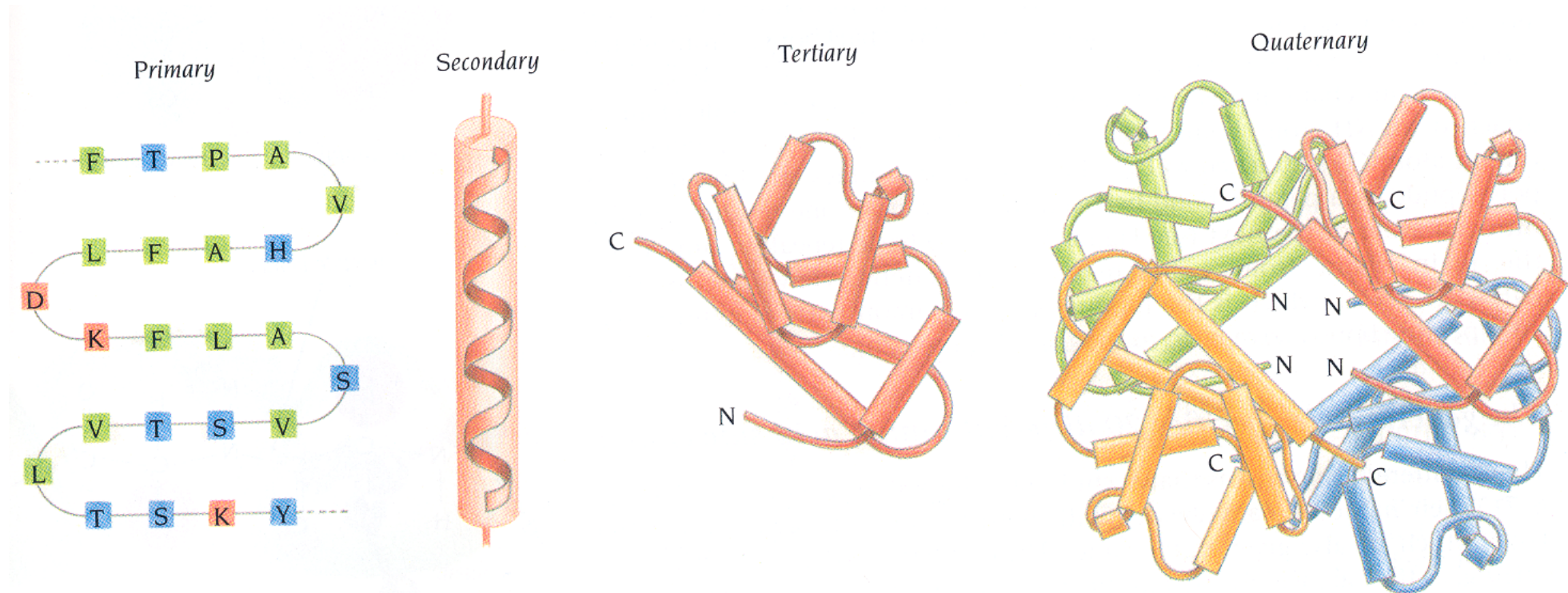
The genetic code

Standard genetic code									
1st base	2nd base								3rd base
	U		C		A		G		
U	UUU	(Phe/F) Phenylalanine	UCU	(Ser/S) Serine	UAU	(Tyr/Y) Tyrosine	UGU	(Cys/C) Cysteine	U
	UUC		UCC		UAC		UGC		C
	UUA		UCA		UAA ^[B]	Stop (Ochre)	UGA ^[B]	Stop (Opal)	A
	UUG		UCG		UAG ^[B]	Stop (Amber)	UGG	(Trp/W) Tryptophan	G
C	CUU	(Leu/L) Leucine	CCU	(Pro/P) Proline	CAU	(His/H) Histidine	CGU	(Arg/R) Arginine	U
	CUC		CCC		CAC		CGC		C
	CUA		CCA		CAA	(Gln/Q) Glutamine	CGA		A
	CUG		CCG		CAG		CGG		G
A	AUU	(Ile/I) Isoleucine	ACU	(Thr/T) Threonine	AAU	(Asn/N) Asparagine	AGU	(Ser/S) Serine	U
	AUC		ACC		AAC		AGC		C
	AUA		ACA		AAA	(Lys/K) Lysine	AGA	(Arg/R) Arginine	A
	AUG ^[A]	(Met/M) Methionine	ACG		AAG		AGG		G
G	GUU	(Val/V) Valine	GCU	(Ala/A) Alanine	GAU	(Asp/D) Aspartic acid	GGU	(Gly/G) Glycine	U
	GUC		GCC		GAC		GGC		C
	GUA		GCA		GAA	(Glu/E) Glutamic acid	GGA		A
	GUG		GCG		GAG		GGG		G

Wikipedia

Highly redundant: only 20 (or 21) amino acids formed from $4^3 = 64$ possible combinations.

Protein Structure: primary to quaternary



Durbin et al., Cambridge University Press

Structure is determined by the **primary sequence** and their **physico-chemical interactions** in the medium.

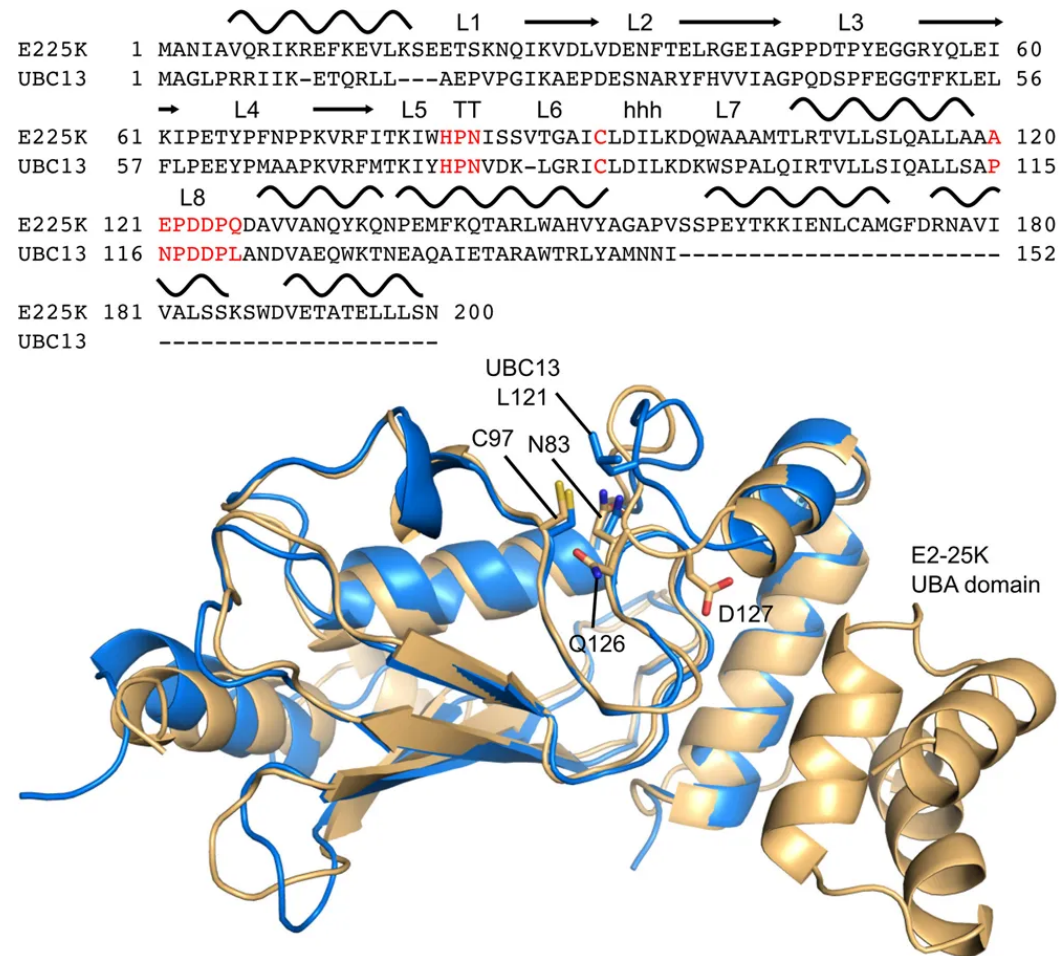
Structure determines functionality.

Mutations

- **Mutation:** Heritable change in the DNA sequence. Occur due to exposure to **ultra violet radiation** or other **environmental conditions**.
- **Point mutations:** within a single gene.
 - **substitution** (change of one nucleotide),
 - **insertion** (addition of nucleotides),
 - **deletion**.
- Mutations occur **spontaneously** during **DNA replication**.
- The rate of mutation is increased by **mutagens**.
- Mutagens: Physical (UV-, X-rays or heat), or chemical (molecules misplace base pairs / disrupt helical shape of DNA).
- Mutations are responsible for **inherited disorders & diseases**.
- Mutations are the source of **phenotypic variation**
⇒ **new species** and **adaption** to environmental conditions.

Sequence Comparison: Motivation

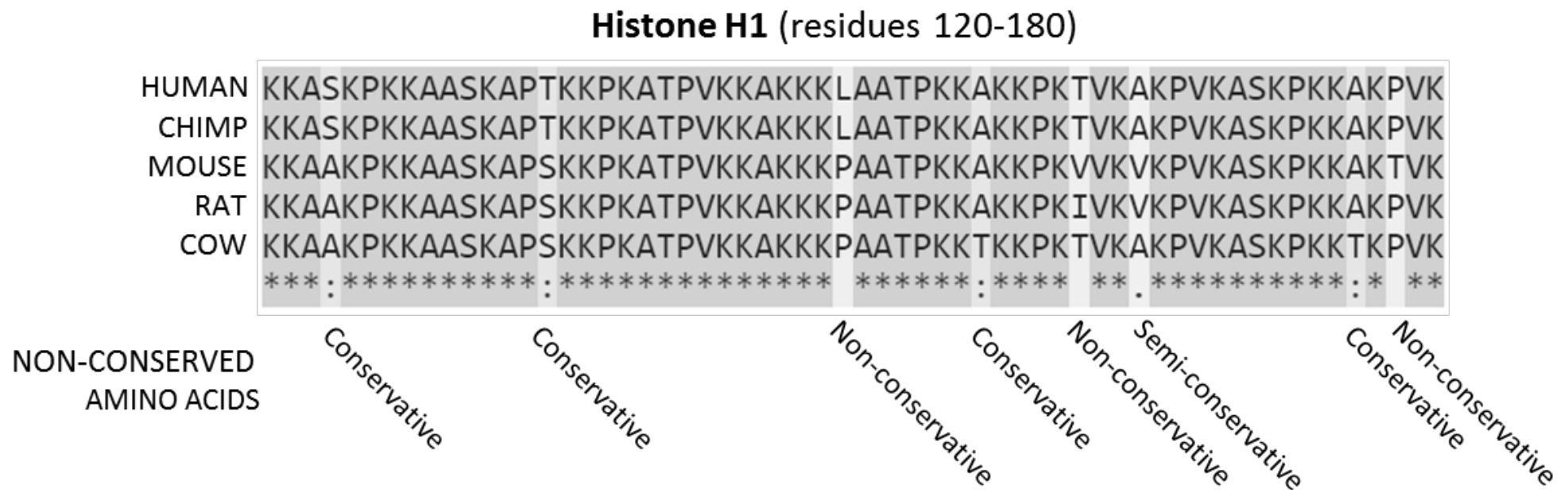
Basic idea: **similar sequences $\rightsquigarrow$ similar proteins.**



Comparing sequences

Theory: during evolution **mutations** occurred, creating differences between families of contemporary species.

Comparing two sequences: looking for **evidence** that they have **diverged from a common ancestor** by a **mutation process**.



By

Sequence Alignment

Informal definition:

Alignment of sequences $x = x_1 \dots x_n$ and $y = y_1 \dots y_m$:

- (i) **insert spaces**,
- (ii) place resulting sequences **one above the other** so that every character or space has a counterpart.

Example: ACBCDDDB and CADBDAD. Possible alignments:

A	C	-	-	B	C	D	D	D	B
-	C	A	D	B	-	D	A	D	-
-	A	C	B	C	D	D	D	B	
C	A	D	B	D	A	D	-	-	

Optimal Alignment

Given: two sequences x and y over alphabet $\mathcal{A}$.

$\mathcal{A} = \{A, G, C, T\}$ (DNA)

$\mathcal{A} = \{A, R, N, D, C, Q, E, G, H, I, L, K, M, F, P, S, T, W, Y, V\}$ (proteins)

Formalizing **optimality of an alignment**: define

- the costs for **substituting** a letter by another letter
 $\Rightarrow$ **substitution matrix**;
- the costs for **insertion** $\Rightarrow$ **gap penalties**.

The Scoring Model

- **Idea:** assign a score to each alignment, choose best one.
- **Additive** scoring scheme: Total score = sum of all scores for pairs of letters + costs for gaps.
Implicit assumption:
Mutations at different sites have occurred **independently**.
(In most cases) reasonable for DNA and protein sequences.
- **All** common algorithms use **additive scoring schemes**.
- Modeling dependencies is possible, but at the price of significant computational complexities.

Substitution Matrices

- **Expectation:**

Identities in real alignments are more likely than by chance.

- Derive score for aligned pairs from a **probabilistic model**.

- **Score:** relative likelihood that two sequences are evolutionary related as opposed to being unrelated

$\rightsquigarrow$ **score = ratio of probabilities.**

- **First assumption:** Ungapped alignment, $n = m$.

- **R : Random model:**

Letter a occurs **independently** with some frequency q_a

$\Rightarrow \text{Pr}(\text{two sequences}) = \text{product of probabilities for each letter:}$

$$P(x, y|R) = \prod_i q_{x_i} \prod_i q_{y_i}.$$

Substitution Matrices

- M (**match**): aligned pairs occur with **joint probability**

$$P(x, y|M) = \prod_i p_{x_i y_i}$$

- Ratio $\rightsquigarrow$ “**odds ratio**”:

$$\frac{P(x, y|M)}{P(x, y|R)} = \prod_i \frac{p_{x_i y_i}}{q_{x_i} q_{y_i}}$$

- To arrive at an **additive** scoring system $\rightarrow$ **log-odds ratio**:

$$S = \sum_i \log \left(\frac{p_{x_i y_i}}{q_{x_i} q_{y_i}} \right) = \sum_i s(x_i, y_i)$$

- $s(a, b)$: log-likelihood ratio of pair (a, b) occurring as an **aligned pair** as opposed to an **unaligned pair** $\rightsquigarrow$ **substitution matrix**.

BLOSUM62 substitution matrix

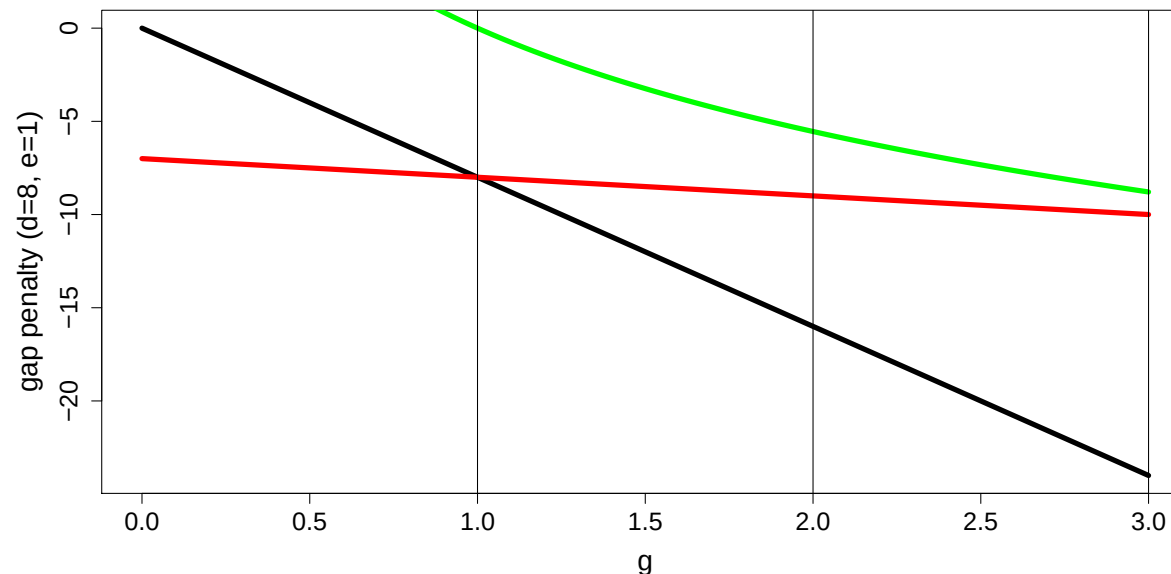
Ala	4																			
Arg	-1	5																		
Asn	-2	0	6																	
Asp	-2	-2	1	6																
Cys	0	-3	-3	-3	9															
Gln	-1	1	0	0	-3	5														
Glu	-1	0	0	2	-4	2	5													
Gly	0	-2	0	-1	-3	-2	-2	6												
His	-2	0	1	-1	-3	0	0	-2	8											
Ile	-1	-3	-3	-3	-1	-3	-3	-4	-3	4										
Leu	-1	-2	-3	-4	-1	-2	-3	-4	-3	2	4									
Lys	-1	2	0	-1	-3	1	1	-2	-1	-3	-2	5								
Met	-1	-1	-2	-3	-1	0	-2	-3	-2	1	2	-1	5							
Phe	-2	-3	-3	-3	-2	-3	-3	-3	-1	0	0	-3	0	6						
Pro	-1	-2	-2	-1	-3	-1	-1	-2	-2	-3	-3	-1	-2	-4	7					
Ser	1	-1	1	0	-1	0	0	0	-1	-2	-2	0	-1	-2	-1	4				
Thr	0	-1	0	-1	-1	-1	-1	-2	-2	-1	-1	-1	-1	-2	-1	1	5			
Trp	-3	-3	-4	-4	-2	-2	-3	-2	-2	-3	-2	-3	-1	1	-4	-3	-2	11		
Tyr	-2	-2	-2	-3	-2	-1	-2	-3	2	-1	-1	-2	-1	3	-3	-2	-2	2	7	
Val	0	-3	-3	-3	-1	-2	-2	-3	-3	3	1	-2	1	-1	-2	-2	0	-3	-1	4
	Ala	Arg	Asn	Asp	Cys	Gln	Glu	Gly	His	Ile	Leu	Lys	Met	Phe	Pro	Ser	Thr	Trp	Tyr	Val

Wikipedia

Gap penalties

Gap penalty types for a gap of length g :

- **Linear**: $\gamma(g) = -gd$, with d being the **gap weight**.
- **Affine**: $\gamma(g) = -d - (g - 1)e$,
gap-open penalty d , gap-extension penalty e . Usually $e < d$.
- **Convex**: e.g. $\gamma(g) = -d \log(g)$. Each additional space contributes less than the previous space.



Global Alignment: Needleman-Wunsch algorithm

The Global Alignment problem:

INPUT: two sequences $x = x_1 \dots x_n$ and $y = y_1 \dots y_m$.

TASK: Find optimal alignment for linear gap penalties $\gamma(g) = -gd$.

Let $F(i, j)$ be the optimal alignment score of the **prefix sequences** $x_{1\dots i}$ and $y_{1\dots j}$. A zero index $i = 0$ or $j = 0$ refers to an **empty sequence**. $F(i, j)$ has following properties:

Base conditions:

$$F(i, 0) = \sum_{k=1}^i -d = -id$$
$$F(0, j) = \sum_{k=1}^j -d = -jd, \quad F(0, 0) = 0.$$

Recurrence relation: for $1 \leq i \leq n, 1 \leq j \leq m$:

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) - d \\ F(i, j-1) - d \end{cases}$$

Tabular Computation of Optimal Alignment

Starting from $F(0,0) = 0$, fill the whole matrix $(F)_{ij}$:

for $i = 0$ or $j = 0$, calculate new value from left-hand (upper) value.

$F(0,0)$ 0	$F(1,0)$ $-d$ $\rightarrow$	$F(2,0)$ $-2d$ $\rightarrow$	
$F(0,1)$ $-d$ $\downarrow$			
$F(0,2)$ $-2d$			

for $i, j \geq 1$, calculate the bottom right-hand corner of each square of 4 cells from one of the 3 other cells:

	$F(i-1,j-1)$ $+s(x_i,y_j)$	$F(i,j-1)$ $-d$	
	$F(i-1,j)$ $-d$	$F(i,j)$	

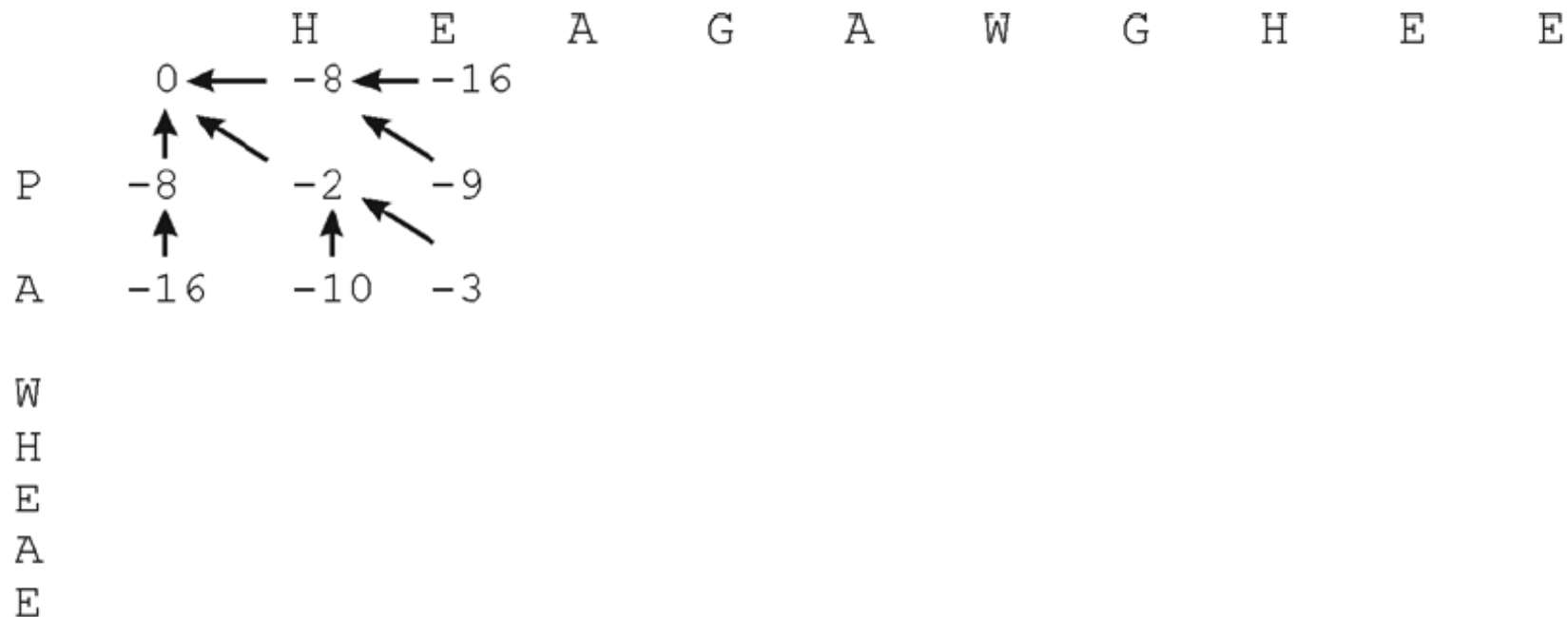
keep a pointer in each cell back to the cell from which it was derived

$\Rightarrow$ **traceback pointer**.

Global Alignment: Example

$x = \text{HEAGAWGHEE}$, $y = \text{PAWHEAE}$. Linear gap costs $d = 8$.

Scoring matrix: BLOSUM50



Durbin et al., Cambridge University Press

Example: traceback procedure

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-42	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	3	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	-5	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	1

H E A G A W G H E - E
 - - P - A W - H E A E

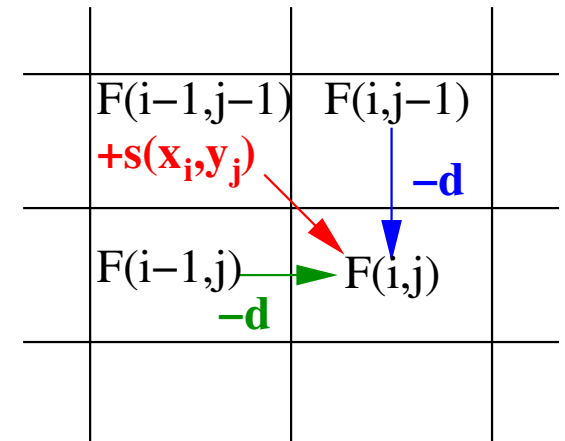
Add pair of symbols: $\nwarrow: (x_i, y_j)$, $\uparrow: (-, y_j)$, $\leftarrow: (x_i, -)$

Time and Space Complexity

Theorem. *The time complexity of the Needleman-Wunsch algorithm is $O(nm)$. Space complexity is $O(m)$, if only $F(x, y)$ is required, and $O(nm)$ for the reconstruction of the alignment.*

Proof:

Time: when computing $F(i, j)$, only cells $(i - 1, j - 1)$, $(i, j - 1)$, $(i - 1, j)$ are examined
 $\rightsquigarrow$ constant time. There are $(n + 1)(m + 1)$ cells
 $\rightsquigarrow$ $O(nm)$ **time complexity.**



Space : row-wise computation of the matrix: for computing row k , only row $k - 1$ must be stored $\rightsquigarrow$ $O(m)$ **space.**

Reconstructing the alignment: all traceback pointers must be stored
 $\rightsquigarrow$ $O(nm)$ **space complexity.**